Chapter 4

by Lorraine Gaudio

Version February 03, 2026

BOISE STATE UNIVERSITY

Contents

1 Overview **2**

2 R Notebooks—Part 2 **3**
2.1 Default Text 3

3 Transparency **5**

4 Vector Metadata **7**
4.1 Factor Vectors (Categorical Data) 7
4.2 Named Vectors 10
4.3 Indexing Vectors 11

5 Logical Vectors **13**

6 Summary **14**

7 Chapter Terms **15**

8 Practice Space **18**
8.1 Task 1 18
8.2 Task 2 19
8.3 Task 3 19
8.4 Task 4 19
8.5 Task 5 20

9 References **21**

1. Overview

Now that you can work in an R Notebook and write usable memos, Chapter 4 focuses on a higher standard: transparency. In research, "correct code" is not enough. You need to document where help came from (peers, the internet, AI, your instructor), what you learned from it, and how you verified the result. Done well, transparency protects you: it shows academic integrity, makes your work reproducible, and prevents you from accidentally claiming understanding you don't yet have.

This chapter also finishes the "vector metadata" toolkit you'll need before we move into datasets. You will learn factor vectors (categorical data) and why levels matter—because many research variables are categories (condition, group, gender identity options, treatment type, survey responses). If your categories are mishandled, summaries and models can be misleading even when the code runs.

Then you'll learn named vectors and indexing, which are the mechanics behind selecting specific values. This is not "extra." Indexing is how you pull out what you need, and it becomes the backbone of filtering, recoding, and subsetting once we start working with data frames. The point of this chapter is control: you can label data intentionally, select what you mean to select, and document your process so another person can reproduce it.

By the end of Chapter 4, you should be able to use transparency practices responsibly and confidently work with categorical data and indexing.

In Chapter Four you will learn how to:

- □ Practice writing and saving an R Notebook.
- □ Run code from a code chunk.
- □ Perform arithmetic operations on numeric vectors.
- □ Practice transparency in memos
- □ Identify what kind of data an object is using class()
- □ Explain how R coerces mixed□ type vectors.
- □ Build logical vectors.

2. R Notebooks—Part 2

In chapter 3, you learned how to use an R Notebook (.Rmd) as a "reproducible thinking" document: code + output + memo-style writing in one place, in order.

Here's what you can do now:

1. **Create and save an R Notebook** (File > New File > R Notebook) and store it in your course folder.
2. **Recognize the YAML header** at the top of the file and edit the title.
3. **Insert code chunks** (Insert button, shortcut, or typing the fences) and understand that R code only runs inside chunks.
4. **Run chunks** using the play button, Run menu, or keyboard shortcuts and see output directly below.
5. **Memo correctly**:

- Inside a code chunk: text must start with # (otherwise R tries to run it).
- Outside a code chunk: you write normal paragraphs (no # needed).
- Use the notebook to document your learning process (goal, verify, memo, explain), including keeping errors visible (commented out) and explaining your fixes.

An R Notebook is only useful if a human can read it. In this chapter, you'll learn the minimum Markdown skills needed to produce a clean, graded-ready notebook: a date in the **YAML Header**, a clear outline using headings, and a successful knit to HTML.

2.1 Default Text

☐ Please open a new R Notebook in your course folder named chapter4_notes.Rmd.

When you create a new R Notebook or R Markdown file, RStudio provides a template. It is designed to demonstrate the three core components of a notebook:

- The YAML Header: The block of text at the very top, enclosed by three dashes (—). This tells R how to "knit" (render) your document.
- Markdown Text: The white background text (like "This is an R Markdown Notebook"). This shows you how to write regular prose.
- Code Chunks: The gray boxes containing code (like plot(cars)). This shows you how to embed executable R code.

2.1.1 Clear the Canvas

To start fresh, delete all the default text in the R Notebook so you have a blank canvas to work with.

Keep the Header Do not delete the text at the very top between the — lines. This is your metadata (Title, Author, Output format). If you delete this, your file might not run correctly.

Delete the Body You can safely delete everything below that second set of three dashes (—).

1. Place your cursor on the line immediately following the second —.
2. Highlight everything all the way to the bottom of the document.
3. Press Delete or Backspace.

3. Transparency

Science isn't just about numbers; it's about values that guide how and why we do our work. Researchers , policy makers and the general public must to trust scientific expertise for this work to have meaning (Resnik & Elliott, 2023). Transparency in academic work promotes trust. **Transparency** is disclosing methods, materials, assumptions, values, and interests (Resnik & Elliott, 2023) so another person can understand what you did and reproduce it.

Three values inside of transparency.

1. **Honesty** is reporting what actually happened, including shortcuts, mistakes, and unexpected results. You record what you tried even when it failed to produce the results you anticipated. You state what you don't know yet. You do not present outside work as your own. For example, you do not present AI outputs as your own. LLMs produce text; they are not conscious "reasoners" that understand your goal the way you do. Being honest includes respecting others' intellectual property by giving credit where credit is due and acknowledging prior work. Researchers must ensure that they properly attribute any contributions made by LLMs to avoid issues of plagiarism and intellectual property infringement (e.g, Thomas, 2023).
2. **Openness** is making your work inspectable so someone else can see what you used and how you got your result. Reproducibility is a key aspect of openness. Creating reproducible work is a core learning outcome for this course. Share the data source, code, and steps taken so someone else can follow your workflow and reproduce your results.
3. **Accountability** means taking responsibility for one's research; responding to criticisms of one's research, including allegations of misconduct; self-correction (carefully scrutinizing research during peer review and after publication; retracting or correcting flawed research); and attempting to reproduce results (Resnik & Elliott, 2023). Transparency practices provide the documentation that makes your assumptions, methods, and tools visible. Memoing around your code is not "extra writing." It is part of doing credible work.

Research journals are running records of your R analysis process. They are an internal document that are invaluable when preparing publications or other reports. Add comments before or near your code explaining:

- □ Goal: The intent of the code and method used (functions, steps, key decisions).
- □ Materials: Compose the code to achieve the goal, answer the question or test your prediction. Keep a record of the algorithms used in documents such as an R Script or R Markdown files.
- □ Assumptions: Document your research question or make a prediction about what the results will be. Keep a record of what you tried, even if it didn't achieve the results you wanted.

- □ Verify: Show or explain how you checked that your results were correct.
- □ Comment: Explain the results and what the output means.
- □ Metacognition: Reflect on the knowledge gained and how it might apply to other work. Record challenges you faced and how you overcame them. State the implications of the analysis or the values that motivate your analysis. Identify anything that could be bias choices.
- □ Resource Cited: Document what resource you used and what you learned from that resource.

Next, you will learn about data types and how to create different kinds of vectors in R. Write memos as you practice R programming.

4. Vector Metadata

In this section you will learn advanced skills that make vectors easier to work with:

1. **Factor Class**: how R stores categorical data using integer codes and labels.
2. **Names**: how to attach labels to vector elements so the vector can act like a simple lookup table.
3. **Indexing**: how to retrieve specific elements using either positions (1st, 2nd, 3rd) or names ("Octarine", "Matrix Green").

4.1 Factor Vectors (Categorical Data)

A **factor** is R's way of storing categorical data. These values that come from a fixed set of categories (for example: "freshman", "sophomore", "junior", "senior"). Factors are common in data analysis because they help R treat categories correctly in tables, summaries, and models.

A factor often looks like a character vector, but it is not the same thing. In fact, object *type* will sometimes surprise you!

4.1.1 Creating Factors

The function factor() is used to create factor vectors from character vectors.

 The SYNTAX (basic):

factor(x = character())

 Create a factor vector named grades that contains the following letter grades: "A", "B", "C", "A", "B", "A" using the factor() function.

```
# Type this in your R Notebook
grades <- c("A", "B", "C", "A", "B", "A")
grades <- factor(grades)
```

 The object grades is overwritten as a factor using the factor() function because the same name is used in the assignment process grades <-.

```
# Check the grades factor
mode(grades)
class(grades)
print(grades)
```

 The output shows that the grades object is of type "numeric" and class "factor." The printed output displays the levels of the factor, which are the unique categories present in the vector.

4.1.2 Levels and Ordering

Even though the elements are text strings, the underlying type is stored as integer. Specifically, mode() often reports numeric while typeof() reports the precise storage type as integer. **Levels** are the distinct categories a factor can take, ordered in a specific way. By default, R usually orders factor levels alphabetically. You can override this default ordering by specifying the levels when creating the factor.

 Create a factor vector named sizes with levels ordered as "Small," "Medium," and "Large" using the factor() function.

```
#  Type this in your R Notebook
sizes <- c("Medium", "Large", "Small", "Medium", "Large")
sizes <- factor(sizes, levels = c("Small", "Medium", "Large"))
```

```
#  Check the sizes factor
print(sizes)
```

 The output shows the sizes factor with levels ordered as "Small," "Medium," and "Large," as specified in the factor() function.

 Memo Example

 Notice: The student (1) confirmed the original object's class (character) and then created a factor version, (2) checked multiple diagnostics (mode(), typeof(), and levels()) to investigate what changed, (3) wrote a memo that pinpointed the exact contradiction ("stored as numbers but prints words") instead of hand-waving, and (4) used an AI tool to research the mechanism and then summarized the explanation accurately in their own words (integer codes + levels labels).

This is a high-quality memo because it captures confusion, gathers diagnostic evidence, and then resolves the confusion with a correct explanation.

Another type of attribute that vectors can have is names. Let's look at named vectors more closely.

4.1.3 Data Coercion

A atomic vector means that all elements must be of the same type. If you try to mix types, R will coerce (convert) them to a common type based on a hierarchy of flexibility.

 What happens if you mix numbers and characters?

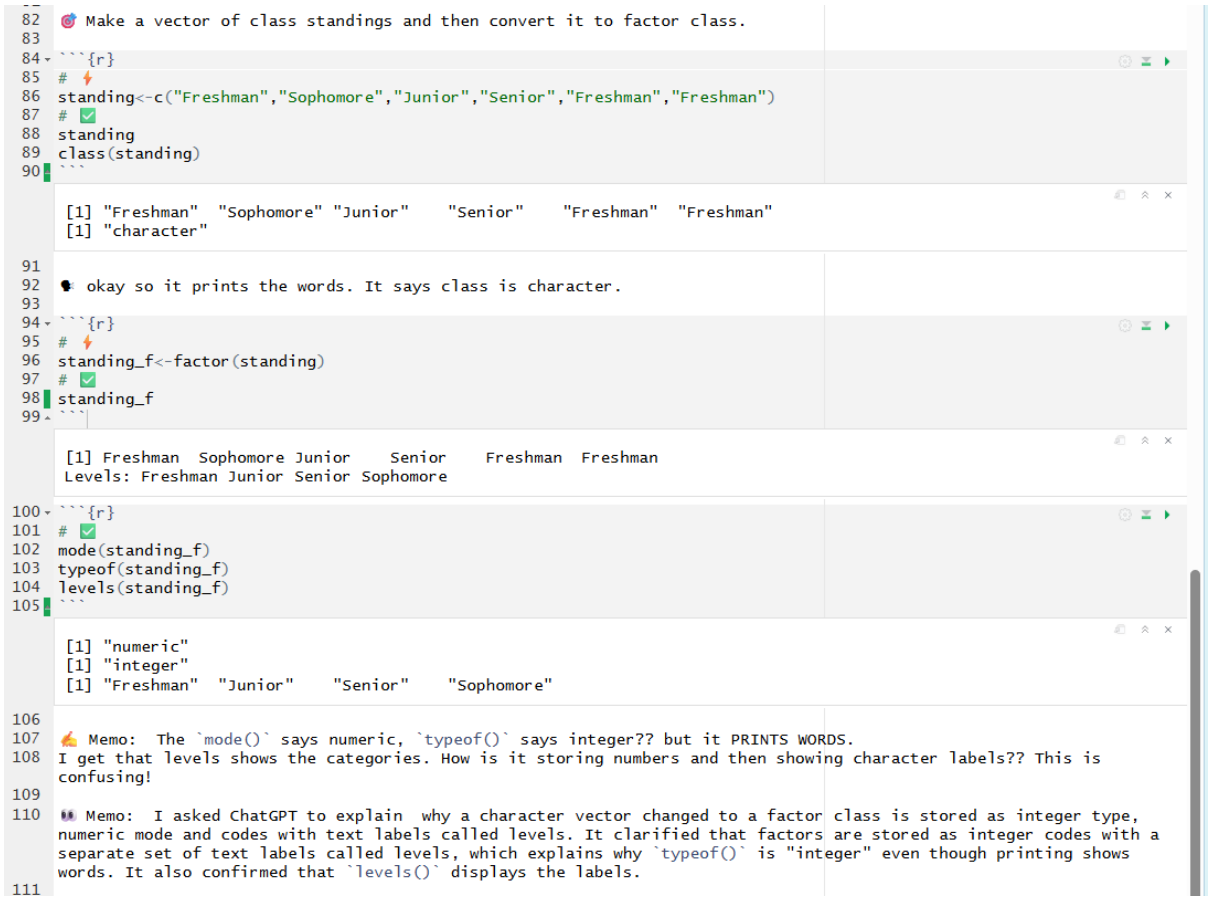

Figure 4.1: R Notebook creating a character vector of class standings, converting it to a factor, and showing levels() plus mode()/typeof() outputs; memo questions why the factor prints words but stores integer codes, with a follow-up memo summarizing the explanation (integer codes + levels labels).

```r
# □ Type this in your R Notebook
mixed <- c("candy", 4, "book", 92)
# □ Check the mixed vector
class(mixed)          # character: all elements coerced to character
```

□ A vector can only hold *one* type. Mixing types triggers *coercion* to the most flexible type (logical → numeric → character).

4.1.3.1 Converting Between Types

You can explicitly convert between types using functions like as.numeric(), as.character(), and as.logical().

The as.numeric() function tries to convert characters to numbers. If it can't, it returns NA (not available).

□ The SYNTAX (basic):

as.numeric(x)

□ Try converting a character vector of numbers to numeric using the as.numeric() function.

```r
as.numeric(c("1", "2", "3")) # character → numeric: text converts to numbers
```

□ The output shows the numeric values 1, 2, and 3 successfully converted from character strings.

□ Try converting the mixed vector to numeric using the as.numeric() function.

```r
# □ Type this in your R Notebook
as.numeric(mixed)          # character → numeric: text converts to NA
```

□ The output shows NA for the character elements "candy" and "book" because they cannot be converted to numeric values. The numeric elements 4 and 92 are successfully converted.

4.2 Named Vectors

Vectors that carry a names attribute so each element can be referred to by a label (a string), not just by position. R implements this via the names() accessor/replacement, which sets the "names" attribute on vectors.

□ Create a character vector named nerdy_palette that contains the following color names and their corresponding hexadecimal color codes using the c() function.

```r
# □ Type this in your R Notebook, Fill in the blanks
nerdy_palette <- c("Octarine" = "#7F00FF", "TARDIS Blue" = "#003B6F", "Rebeccapurple" = "#663399", "H
```

```
# □ Check the nerdy_palette vector
print(nerdy_palette)
names(nerdy_palette)
```

□ The nerdy_palette vector acts as a look-up table, allowing you to retrieve color hex codes by their assigned color names.

4.2.1 Object names vs Element names

There are two different naming ideas here.

1. **Object name** is the variable name nerdy_palette that you use to refer to the entire vector.
2. **Element names** are the labels assigned to each element within the vector (e.g., "Octarine", "TARDIS Blue").

Element names can be used in indexing to access specific elements of the vector.

4.3 Indexing Vectors

The **bracket operator** [is the standard way to extract (or replace) parts of a vector. The basic pattern is x[i], where x is the vector and i is the index (or indices) of the elements you want to extract. The [operator can select one element or many and it returns an object of the same type as x.

4.3.1 Indexing by Name

When a vector has names, you can use those names to index into the vector. **Character indexing** uses the names of the elements to retrieve specific values.

□ Use character indexing to return the hexadecimal color code for "Octarine."

```
# □ Type this in your R Notebook
nerdy_palette["Octarine"]
```

□ The output shows the hexadecimal color code for "Octarine" retrieved from the nerdy_palette vector using its name.

Character indexing required exact matching on names. If you ask for a name that doesn't exist, you get NA for that element.

□ Use character indexing to retrieve the color codes for "Octarine," "TARDIS Blue," and "NotA-Color" from the nerdy_palette vector.

```
# □ Type this in your R Notebook
nerdy_palette[c("Octarine", "TARDIS Blue", "NotAColor")]
```

 The output shows the hexadecimal color codes for "Octarine" and "TARDIS Blue," and NA for "NotAColor," which does not exist in the nerdy_palette vector.

4.3.2 Indexing by Position

Positional indexing uses the numeric positions of elements in the vector to retrieve specific values.

 Use positional indexing to return the first and third color codes from the nerdy_palette vector.

```
#  Type this in your R Notebook
nerdy_palette[c(1, 3)]
```

 The output shows the hexadecimal color codes for Octarine and Rebeccapurple in the nerdy_palette vector.

We will return to indexing in later chapters when we learn about data frames.

5. Logical Vectors

REVIEW

Logical operators include:

- > greater than
- < less than
- == equal to
- >= greater than or equal to
- <= less than or equal to
- != not equal to

 Create a new vector called smallerthat contains only values in number_list that are less than or equal to 20.

```
# Type this in your R Notebook
number_list <- c(10, 25, 5, 30, 15, 40)
smaller <- number_list[number_list <= 20]
```

```
# Compare the vectors
print(number_list)
print(smaller)
```

 The output shows the values in the number_list vector that are less than or equal to 20, which are 10, 5, and 15.

---

6. Summary

In this chapter, you learned to create an R Notebook, rename and save it correctly, and use code chunks to run R code with output shown directly beneath your work. You also practiced essential vector skills: doing arithmetic on numeric vectors (including recycling), checking an object's class, combining text with paste(), creating factors, understanding how mixed-type vectors trigger coercion, creating named vectors, and indexing by position or name. Finally, you connected these technical skills to transparency by writing memos that clearly state your goal, show verification, explain results, and document any outside resources you used.

7. Chapter Terms

Accountability: Taking responsibility for the results you report by verifying them, responding to questions or critiques, and correcting errors when found. In practice, this means you keep a clear record (e.g., in an R Markdown/Notebook or script) that shows how you checked your work and why you made your choices.

Backtick: The ‘ character (above the Tab key) used for code-related formatting and quoting. In **Markdown**, single backticks mark *inline code* and triple backticks create *fenced code blocks*; in **R Markdown**, backticks also mark *inline R code* (‘r …‘), including inside YAML fields like date: “‘r Sys.Date()‘”, which gets evaluated when you render the document. In R, backticks can also quote non-syntactic names (e.g., names with spaces) so they can be used safely in code.

Bracket Operator: The square-bracket indexing operator [used to extract or replace parts of an object in R (for example, x[i]). For vectors, it selects one or more elements and returns the same kind of object as x (a vector).

Character Indexing: Indexing a named vector using element names (character strings) instead of numbers. Names must match exactly; a missing name returns NA.

Character String: A single piece of text in R, stored as a character value and written in quotes (for example, “My favorite color is yellow!” or “Who stole the cookie from the cookie jar?”).

Class class(): A function that reports the object’s class, which tells R how to interpret the object and which methods to use when functions are applied to it. For example, class(3.14) returns "numeric", and class(mtcars) returns "data.frame".

Code Chunks: Fenced blocks in an R Markdown/Notebook document where R code is written and executed, usually starting with “‘{r}. Only code inside a chunk runs, and its output (including plots and tables) appears directly below the chunk when executed.

Element Names: The individual labels assigned to elements inside a named vector (for example, "Octarine" or "TARDIS Blue"). Element names let you index by label (e.g., nerdy_palette["Octarine"]) to retrieve specific values.

Factor Class: An R object class for categorical data, where each value is stored as an integer code that points to a category label. Factors help R handle categories correctly in tables, plots, and statistical models, even though they may print like text.

Factor Levels: The distinct category labels a factor can take (for example, "freshman", "sophomore", "junior", "senior"). Levels define the set and order of categories, and each factor value is internally stored as the index of its level.

Honesty: Accurately reporting what you did in R—your actual steps, mistakes, revisions, and uncertainty—without presenting outside work as your own. This includes clearly attributing code, ideas, and any AI assistance, and being able to explain the code you submit.

Logical Operators: Operators that create or combine logical elements. The most common are comparison operators—==, !=, <, >, <=, >=—which return TRUE/FALSE, and Boolean operators—! (NOT), & / | (AND/OR applied element-by-element), and && / || (AND/OR that evaluate only the first element, mainly for if conditions).

Logical Vector: A vector that stores Boolean elements: TRUE or FALSE (and sometimes NA for missing). Logical vectors are commonly produced by comparisons (for example, x > 0) and are heavily used for filtering and conditional code

NA: A special constant that marks a missing value ("Not Available")—meaning the data point is unknown or not recorded. R also provides typed missing values like NA_integer_, NA_real_, NA_character_, and NA_complex_ to match the underlying data type.

Named Vectors: Vectors that have labels attached to their elements via the names() attribute, so you can refer to values by name instead of only by position. Named vectors are useful as simple lookup tables (for example, retrieving a hex code by a color name).

Object Name: The variable name you assign with <- that refers to the entire object in your environment. You use the object name to print, modify, or pass the whole object to functions.

Openness: Making your analysis inspectable and reproducible by sharing the inputs and decisions that produced your results (data source, packages, code, and key assumptions). In this course, openness means you provide others with enough details so thay are able to rerun your work and get the same outputs.

Package: An installable collection of R code—functions, documentation, and sometimes datasets—that adds new features beyond base R. Packages are commonly installed from CRAN and then loaded with library() so you can use their tools in your session.

Positional Indexing: Indexing a vector using numeric positions (for example, x[1] for the first element or x[c(1, 3)] for the first and third). Positions are based on the current order of the vector.

R Markdown: A document format (.Rmd) that mixes plain-text writing (Markdown) with embedded R code chunks, so you can generate reports that combine narrative, code, and computed output. You can render an R Markdown file into finished formats like HTML, PDF, or Word.

R Notebook: An .Rmd (R Markdown) document that lets you run code in chunks and see the results (output, tables, plots) immediately below each chunk. It's designed for reproducible work because it keeps your code, results, and explanatory notes together in the same file.

Research Journal: A running record of your R analysis process. This inlcudes what question you asked, what code you tried (including failed attempts), what outputs you got, and what decisions you made along the way. In practice, it's maintained in an R script or R Markdown/Notebook with dated notes, comments, and citations so you (or someone else) can retrace, verify, and reproduce the work.

Transparency: The practice of clearly disclosing your methods, materials, assumptions, tools, values, and sources so another person can follow your workflow, reproduce your results, and evaluate your claims. In this course, transparency supports explanatory power (you can explain what your code does and why) and originality/accountability (you produce and attribute work honestly, including documenting if and how you used AI).

Vectorization: R's behavior of applying a function to an entire vector at once, producing results element-by-element without writing an explicit loop. For example, paste(first, last) combines the first elements together, then the second, then the third, returning a vector of full names.

YAML Header: The metadata section at the top of an R Markdown or Quarto document, written between --- lines, that controls document settings like the title, author, date, output format (HTML/PDF/Word), and options for rendering.

8. ⛒ Practice Space

Fill in the blanks to complete the code.

Before you begin, make sure you have created a new R Notebook in the Source pane and saved it as chapter4_practice.Rmd in your course folder.

Remember to document your **tenacity** on a memo if you encounter any errors while completing the tasks. Document how you troubleshot the error and what resource(s) you used to help you resolve the issue (**transparency**).

R Notebooks can look "fine" even when they are not reproducible because your Environment may still contain objects from earlier work. Start with a clean environment and run everything in order.

8.1 Task 1

□ **Indexing named vectors**

□ Use character indexing to retrieve the hexadecimal color codes for "Matrix Green" and "Command Red" from the nerdy_palette vector.

```
# □ Type this in your R Notebook
nerdy_palette <- c(
  "Octarine" = "#7F00FF",
  "TARDIS Blue" = "#003B6F",
  "Rebeccapurple" = "#663399",
  "Hooloovoo" = "#4F97A3",
  "Cosmic Latte" = "#FFF8E7",
  "Vantablack" = "#000000",
  "Matrix Green" = "#03A062",
  "Source Engine Magenta" = "#FF00FF",
  "Command Red" = "#CC0000",
  "Impossible Green" = "#4D9E68"
)
```

```
# □ Type this in your R Notebook, Fill in the blanks
nerdy_palette[c(_____, _____)]
```

□ The output shows the hexadecimal color codes for "Matrix Green" and "Command Red" retrieved from the nerdy_palette vector using their names.

□ *Memo: articulate your learning process, connect ideas, or discuss challenges.*

8.2 Task 2

Indexing by position

Use positional indexing to retrieve the 2nd, 4th, and 6th color codes from the nerdy_palette.

```
# Type this in your R Notebook, Fill in the blanks
_____[c(_, _, _)]
```

The output shows the hexadecimal color codes for the 2nd, 4th, and 6th colors in the nerdy_palette vector.

Memo: articulate your learning process, connect ideas, or discuss challenges.

8.3 Task 3

Logical filtering + indexing practice

Use the change vector to complete the following:

1. Create a new vector positive_change that contains only values in change that are greater than zero.
2. Create a new vector last_two that contains the last TWO elements of change (use numeric positions, not names).

```
# Type this in your R Notebook, Fill in the blanks
change <- c(-2, 0, 5, 8, -1, 3)
positive_change <- change[change _ 0]
last_two <- change[c(___, ___)]
```

```
# Check the outputs
print(positive_change)
print(last_two)
```

The output shows _____.

Memo: articulate your learning process, connect ideas, or discuss challenges.

8.4 Task 4

Reflection

In one sentence, explain why numeric values may be coerced to character if you mix them inside a single vector.

Your explanation here _____

8.5 Task 5

 Step 1: Sweep your Environment (start clean).
Click the **broom icon** in the Environment pane to clear objects.

 Step 2: Run everything top-to-bottom.
Use **Run → Run All** to execute every chunk in order.

 Step 3: Save and submit.
After your notebook runs without errors, save your notebook (.Rmd) for submission to Canvas, as directed.

9. References

Resnik, D. B., & Elliott, K. C. (2023). Science, values, and the new demarcation problem. Journal for General Philosophy of Science, 54(2), 259-286.

Thomas, P. A. (2023). Wikipedia and large language models: perfect pairing or perfect storm?. Library Hi Tech News, 40(10), 6-8.